



Figure 1: Dimensions of technical debt

As one can imagine, its software developers are expected to implement features faster. In such a case, the developers may not have the opportunity or time to properly assess the impact of their design decisions. As a result, over time, such a collection of individual localised design decisions starts to degrade the structural quality of the software products, thereby contributing to the accumulation of design debt.

If such a product were to be developed just once and then no longer maintained, the structural quality would not matter. However, most products are in the market for a long time period and therefore have an extended development and maintenance life cycle. In such cases, the poor structural quality of the software will significantly increase the effort and time required to understand and maintain the software. This will eventually hurt the organisation's interests. Thus, it is extremely important for organisations to monitor and address the structural quality of the software. The work that needs to be invested in the future, to address the current structural quality issues in the software, is design debt.

An interesting question in the context of what constitutes technical debt is whether defects/bugs are a part of this debt. Some argue that defects (at least some of them) originate due to technical debt, and thus are part of technical debt. Their view is that if managers decide to release a software version despite it having many known yet-to-be-fixed defects, these are a part of the technical debt that has been incurred.

However, there are others in the community who argue that defects do not constitute technical debt. Their view is that the main difference between defects and technical debt is that defects are visible to the users while technical debt is largely invisible. We support this stance. In our experience, defects are rarely ignored by the organisation and receive much attention from the development teams. On the other hand, issues leading to technical debt are mostly invisible and tend to receive little or no attention from the development teams. Why does this happen?

This happens because defects directly impact the external quality attributes of the software that are directly visible to the end users. Technical debt, on the other hand, impacts the internal quality of the software system, and is not directly perceivable by the end users of the software.

Organisations value their end users and cannot afford to lose them; thus, defects get the utmost attention while issues related to 'invisible' technical debt are usually deferred or ignored. Thus, from a practical viewpoint, it is better to leave defects out of the umbrella term 'technical debt', so that they can be dealt with separately; otherwise, one would fix defects and mistakenly think that the technical debt has been addressed.

What is the impact of technical debt?

Why is it important for a software practitioner to be aware of technical debt and keep it under control? To understand this, let us first understand the components of technical debt, which is a result of the 'principal' (the original hack or shortcut), and the accumulated interest incurred when the principal is not fixed. The interest component is compounding in nature; the more you ignore or postpone it, the bigger the debt becomes over time. Thus, it is the interest component that makes technical debt a significant problem.

Why is the interest compounding in nature for technical debt? One major reason is that often, new changes introduced in the software become interwoven with the debt-ridden design structure, further increasing the debt. Then, going forward, when the original debt remains unpaid, it encourages or even forces developers to use 'hacks' while making changes, which further compounds the debt.

Jim Highsmith describes how the 'Cost of Change' (CoC) varies with technical debt. A well-maintained software system's actual CoC is close to the optimal CoC; however, with the increase in technical debt, the actual CoC also increases. As previously mentioned, in extreme cases, the CoC can become prohibitively high leading to technical bankruptcy.

Apart from technical challenges, technical debt also impacts the morale and motivation of the development team. As technical debt mounts, it becomes difficult to introduce changes and the team involved with development starts to feel frustrated and annoyed. The frustration is further compounded because the alternative—i.e., repaying the whole technical debt—is not a trivial task that can be accomplished overnight.

It is believed that technical debt is the reason behind software faults in a number of applications across domains, including financing. In fact, a BBC report clearly mentions technical debt as the main reason behind the computer controlled trading error that decimated the balance sheet of Knight Capital, an American global financial services firm in 2012.

What causes technical debt?

The previous section discussed the impact of technical debt in a software system. To pay off the technical debt or to prevent a software system from accruing technical debt, it is